

CWII

Code Wars

Edição 2/2 0 2 4

Fatec **CPS**
Ribeirão Preto Centro Paula Souza

CADERNO DE QUESTÕES

ETAPA ÚNICA

14 de SETEMBRO DE 2024

APOIO



S4T

@ads.fatec Ribeirão Preto

1 Instruções

Este caderno contém 8 problemas – identificados por letras de A até H. com páginas numeradas de 3 até 19. Verifique se seu caderno está completo.

Informações gerais

1. Sobre a competição

- (a) A competição possui duração de 3 horas e 30 min (início às 09h término às 12h30m);
- (b) É permitido a consulta a materiais já publicados anteriormente ao dia da competição
- (c) Não é permitido a comunicação com outras pessoas que não sejam a equipe para tirar dúvidas sobre a competição
- (d) É vedada a comunicação entre as equipes durante a competição, bem como a troca de material de consulta entre elas;
- (e) Cada equipe terá acesso a 1 computador dotado do ambiente de submissão de programas (BOCA), dos compiladores, link-editores e IDEs requeridos pelas linguagens de programação permitidas;
- (f) As linguagens permitidas são: C, C++ (extensão cc), Java e Python 3
- (g) NÃO é permitido o uso de notebooks ou outro tipo de computador ou assistente pessoal;
- (h) Os problemas têm o mesmo valor na correção.

2. Sobre o arquivo de solução e submissão:

- (a) O arquivo de solução (o programa fonte) deve ter o mesmo nome que o especificado no enunciado (logo após o título do problema);
- (b) confirme se você escolheu a linguagem correta e está com o nome de arquivo correto antes de submeter a sua solução;
- (c) NÃO insira acentos no arquivo-fonte.

3. Sobre a entrada

- (a) A entrada de seu programa deve ser lida da entrada padrão (não use interface gráfica);
- (b) Seu programa será testado em vários casos de teste válidos além daqueles apresentados nos exemplos. Considere que seu programa será executado uma vez para cada caso de teste.

4. Sobre a saída

- (a) A saída do seu programa deve ser escrita na saída padrão;
- (b) Não exiba qualquer outra mensagem além do especificado no enunciado.

Problema A

Lanche Rápido

Arquivo fonte: `lanche.{ c | cc | java | py }`

Autor:

O restaurante Lanche Rápido oferece um cardápio variado para seus clientes e precisa de um programa que auxilie no cálculo do valor total de um pedido. Como desenvolvedor, você foi contratado para criar esse programa, que deve calcular o valor da conta com base no código do item e na quantidade solicitada.

Com base na tabela abaixo, escreva um programa que leia o código de um item e a quantidade deste item. A seguir, calcule e mostre o valor da conta a pagar.

Código	Descrição	Preço Unitário (R\$)
1	Cachorro-quente	R\$ 4.00
2	X-Salada	R\$ 4.50
3	X-Bacon	R\$ 5.00
4	Torrada simples	R\$ 2.00
5	Refrigerante	R\$ 1.50

Tabela 1: Tabela de preços do restaurante Lanche Rápido

Entrada

O arquivo de entrada contém dois valores inteiros correspondentes ao código e à quantidade de um item conforme tabela acima.

Saída

O arquivo de saída deve conter a mensagem "Total: R\$ " seguido pelo valor a ser pago, com 2 casas após o ponto decimal.

Exemplo de Entrada 1

3 2

Exemplo de Saída 1

Total: R\$ 10.00

Exemplo de Entrada 2

4 3

Exemplo de Saída 2

Total: R\$ 6.00

Exemplo de Entrada 3

2 3

Exemplo de Saída 3

Total: R\$ 13.50

Esta página foi propositadamente deixada em branco.

Problema B

Discagem Direta à Distância (DDD)

Arquivo fonte: ddd.{ c | cc | java | py }

Autor:

As ligações interurbanas, conhecidas como DDD (Discagem Direta à Distância), permitem a comunicação entre diferentes regiões do Brasil sem a necessidade de intermediários, como as operadoras. O sistema foi implementado para facilitar a ligação entre municípios e estados, padronizando o processo através de códigos numéricos específicos para cada área geográfica. O código DDD consiste em dois dígitos que representam a região de destino, e deve ser discado antes do número do telefone quando a chamada é realizada de uma área diferente daquela onde o telefone está registrado.

Para realizar uma ligação interurbana, o usuário deve discar uma sequência composta por um prefixo, o código da operadora de sua escolha, o código DDD da região de destino, e, por fim, o número de telefone desejado. Por exemplo, para ligar de São Paulo (DDD 11) para o Rio de Janeiro (DDD 21), deve-se discar: 0 + código da operadora + 21 + número do telefone. O sistema DDD torna o processo de discagem mais eficiente e organizado, ao mesmo tempo em que oferece ao usuário a possibilidade de escolher a operadora que preferir, promovendo concorrência e potencialmente reduzindo custos de ligação.

Elabore um programa que leia um número inteiro que representa um código de DDD para discagem interurbana. Em seguida, informe à qual cidade o DDD pertence, considerando a tabela abaixo:

Código DDD	Região de destino
11	São Paulo
21	Rio de Janeiro
31	Belo Horizonte
41	Curitiba
51	Porto Alegre
61	Brasília
71	Salvador
81	Recife
91	Belém

Tabela 2: Tabela de códigos DDD e suas respectivas regiões de destino no Brasil

Entrada

A entrada consiste de um único valor inteiro.

Saída

Imprima o nome da cidade correspondente ao DDD existente na entrada. Imprima *DDD nao cadastrado* caso não existir DDD correspondente ao número digitado.

Exemplo de Entrada 1

11	São Paulo
----	-----------

Exemplo de Saída 1**Exemplo de Entrada 2**

51	Porto Alegre
----	--------------

Exemplo de Saída 2**Exemplo de Entrada 3**

81	Recife
----	--------

Exemplo de Saída 3

Problema C

Notas da UTF

Arquivo fonte: notas.{ c | cc | java | py }

Autor:

A Universidade Tecnológica do Futuro (UTF) é uma instituição de ensino superior reconhecida por sua excelência em pesquisa e inovação. Fundada em 2085, a UTF oferece uma ampla gama de cursos de graduação e pós-graduação, com foco em áreas tecnológicas, análise e desenvolvimento de sistemas, ciências da computação, engenharia e ciências aplicadas. Com um campus moderno e totalmente equipado, a universidade investe fortemente em laboratórios de última geração e em programas de intercâmbio internacional, proporcionando aos seus estudantes uma formação completa e globalizada. A UTF também mantém parcerias estratégicas com grandes empresas do setor tecnológico, promovendo a integração entre academia e indústria, e oferecendo oportunidades de estágio e desenvolvimento profissional para seus alunos.

Para controlar as notas das avaliações a UTF precisa de um programa para ler as notas da primeira e a segunda avaliação de um aluno. O objetivo é calcular e imprimir a média semestral. O programa só deverá aceitar notas válidas (uma nota válida deve pertencer ao intervalo $[0, 10]$). Cada nota deve ser validada separadamente.

No final deve ser impressa a mensagem *novo calculo (1-sim 2-nao)*, solicitando ao usuário que informe um código (1 ou 2) indicando se ele deseja ou não executar o algoritmo novamente, (aceitar apenas os códigos 1 ou 2). Se for informado o código 1 deve ser repetida a execução de todo o programa para permitir um novo cálculo, caso contrário o programa deve ser encerrado.

Entrada

O arquivo de entrada contém vários valores reais, positivos ou negativos. Quando forem lidas duas notas válidas, deve ser lido um valor inteiro X . O programa deve parar quando o valor lido para este X for igual a 2.

Saída

Se uma nota inválida for lida, deve ser impressa a mensagem "nota invalida". Quando duas notas válidas forem lidas, deve ser impressa a mensagem *media =* seguido do valor do cálculo.

Antes da leitura de X deve ser impressa a mensagem *novo calculo (1-sim 2-nao)* e esta mensagem deve ser apresentada novamente se o valor da entrada padrão para X for menor do que 1 ou maior do que 2. A média deve ser impressa com dois dígitos após o ponto decimal.

Exemplo de Entrada 1

```
-3.5
3.5
11.0
10.0
4
1
8.0
9.0
2
```

Exemplo de Saída 1

```
nota invalida
nota invalida
media = 6.75
novo calculo (1-sim 2-nao)
novo calculo (1-sim 2-nao)
media = 8.50
novo calculo (1-sim 2-nao)
```

Exemplo de Entrada 2

```
-1.0
12.0
8.0
7.5
1
9.0
10.0
2
```

Exemplo de Saída 2

```
nota invalida
nota invalida
media = 7.75
novo calculo (1-sim 2-nao)
media = 9.50
novo calculo (1-sim 2-nao)
```

Exemplo de Entrada 3

```
5.0
8.0
1
7.0
9.0
2
```

Exemplo de Saída 3

```
media = 6.50
novo calculo (1-sim 2-nao)
media = 8.00
```

Problema D

Brainfuck

Arquivo fonte: brainfuck.{ c | cc | java | py }

Autor:

Linguagens de programação, times de futebol e religião não se discute. Cada um tem seus favoritos e não admite que o do outro seja melhor (que me perdoem os corinthianos, palmeirenses e são-paulinos). Um grupo de pesquisadores (que não tinha o que fazer) resolveu escrever *uma linguagem de programação ideal* (ideal prá quem, cara pálida?): o **Brainfuck**. Brainfuck é uma linguagem de programação cujo funcionamento é muito parecido com uma máquina de Turing. Essa máquina possui como componentes um vetor de 30000 bytes, indexado de 0 a 29999, e um ponteiro, que guarda uma posição desse vetor. Em cada passo, a máquina realiza uma instrução de acordo com o byte armazenado na posição do vetor indicada pelo ponteiro. Quando esse byte é igual a zero, a execução é terminada. O conjunto de instruções válidas da linguagem é o seguinte:

Instrução	Descrição
>	Incrementa a posição do ponteiro.
<	Decrementa a posição do ponteiro.
+	Incrementa o byte na posição indicada pelo ponteiro.
-	Decrementa o byte na posição indicada pelo ponteiro.
.	Imprime o valor do byte na posição indicada pelo ponteiro.
,	Lê um byte e armazena na posição indicada pelo ponteiro. Se não houver nada que possa ser lido (entrada acabou), armazenar zero.
[	Início do loop: Executa o código delimitado até que o byte na posição indicada pelo ponteiro seja igual a zero.
]	Fim do loop.
#	Imprime os valores das 10 primeiras posições do vetor.

Tabela 3: Instruções da Linguagem Brainfuck

O ponteiro sempre começa com valor 0, assim como todas as posições do vetor. Na descrição de programas na linguagem brainfuck, caracteres diferentes dos descritos acima são ignorados.

Entrada

A entrada é composta de diversas instâncias. O número de instâncias é dado na primeira linha da entrada. Cada instância começa com uma linha em branco. A próxima linha contém uma cadeia de caracteres não-brancos (ou seja, diferentes de espaço em branco e tabulação), que vai conter a entrada para o programa. Ou seja, os comandos de leitura são realizados nessa cadeia. Toda a entrada para o programa está contida em uma única linha. Por fim, a terceira linha contém a descrição do programa. Assim como a segunda linha, esta também não contém caracteres brancos e está inteiramente contida em uma única linha (a separação feita no segundo exemplo de entrada foi feita para evitar o estouro de linha). Tanto a segunda como a terceira linha têm entre 1 e 100000 caracteres.

Saída

Para cada instância, você deverá imprimir um identificador "Instancia k", onde k é o número da instância atual. Na linha seguinte você deve imprimir a saída do código fornecido na entrada. Após cada instância, seu programa deve imprimir uma linha em branco.

Exemplo de Entrada 1

```
2

marrocos
+[>,]<-[+.<-]

nada
+++++++>+++++++>+++++++>++++><<<<->+>.>+.>+++++>.>+>.>+>.<<+>
+++++++>.>+>.>+>.>+>.>+>.>+>.>+>.
```

Exemplo de Saída 1

```
Instancia 1
socorram

Instancia 2
Hello World!
```

Problema E

Detecção dos Sentinelas

Arquivo fonte: sentinela.{ c | cc | java | py }

Autor:

Depois de resgatar *Neo*, *Morpheus* está pilotando o *hovercraft Nebuchadnezzar* de volta a *Zion*, a última cidade humana. Para recapturá-los, as entidades de IA que controlam a **Matrix** enviaram vários robôs sentinelas, robôs gigantes em forma de polvo que podem perseguir e destruir o *hovercraft*.

Os sensores de *Nebuchadnezzar* localizaram vários sentinelas ao redor da nave. *Morpheus* precisa calcular o alcance de seus sistemas de detecção para definir zonas seguras para viagens. Você deve ajudar *Morpheus* escrevendo o código para detectar com precisão a proximidade das sentinelas e soar apropriadamente os alarmes de alerta. As sentinelas de última geração que a **Matrix** projetou podem ser aproximadas por hexágonos regulares (cada lado do hexágono é igual). Sua nave pode ser aproximada por um círculo.

Escreva o código para detectar a sentinela mais próxima da nave. Você pode assumir que todos os hexágonos são orientados da mesma forma com um par de arestas paralelas ao eixo x. A distância deve ser reportada do casco externo do navio até o limite externo do sentinela (não do centro do navio ao centro do sentinela). Se a distância até o sentinela mais próximo for menor que zero (ou seja, o sentinela já tiver violado o casco do navio), a distância deve ser reportada como zero (não um número negativo).

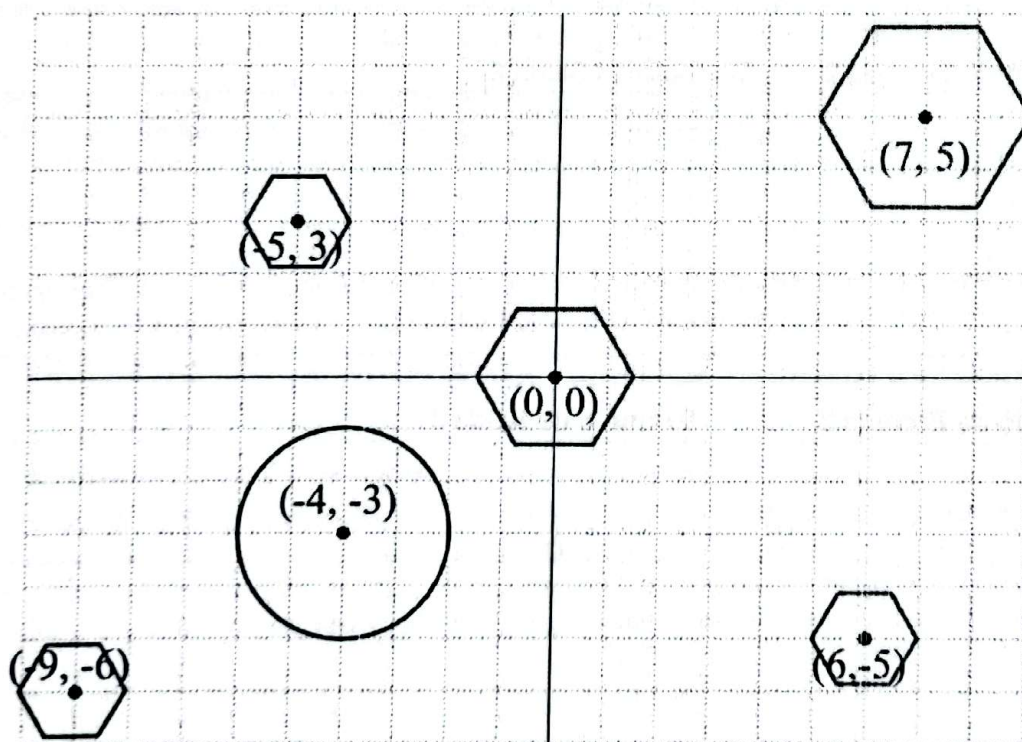


Figura E.1: Cenário de exemplo

Entrada

A primeira linha contém um número inteiro com o número de linhas que você precisa ler. Todos os outros valores devem ser lidos como ponto-flutuante.

A segunda linha contém o centro da posição e raio do navio:

$ship_x$ $ship_y$ $ship_{raio}$

A terceira e as linhas subsequentes contêm os centros e raios das sentinelas:

$sentinel_{ix}$ $sentinel_{iy}$ $sentinel_{iraio}$

Saída

Exemplo de Entrada 1

```
6
-4 -3 2
-9 -6 1
-5 3 0.5
0 0 1.5
6 -5 1
7 5 2
```

Exemplo de Saída 1

A distância do sentinela mais próximo é 1.66506

Exemplo de Entrada 2

```
6
-4 -3 2
-4 -6 1
-5 3 0.5
0 3 1.4
6 -5 1
7 5 2
```

Exemplo de Saída 2

A distância do sentinela mais próximo é 0.133975

Exemplo de Entrada 3

```
2
-4 -3 2
7 5 2
```

Exemplo de Saída 3

A distância do sentinela mais próximo é 9.802

Problema F Helicóptero

Arquivo fonte: helicoptero.{ c | cc | java | py }

Autor:

Como uma das entidades de inteligência artificial (IA) que implementam a Matrix, você é responsável por manter a aparência da realidade física em resposta a eventos na Matrix. Um dia, você está de serviço quando vermes humanos de alguma forma ganham o controle de um helicóptero enquanto tentam escapar de seus agentes. Devido aos seus reflexos humanos limitados, eles jogam o helicóptero na lateral do seu prédio de vidro. Você deve calcular os efeitos de ondulação apropriados resultantes do impacto.

Para modelar a superfície de um edifício, você pode usar uma matriz bidimensional de float (ou double). O valor de cada ponto na matriz indica o quão longe aquela parte da superfície do edifício foi esticada. Você então aplica uma técnica iterativa de diferença finita para modelar os efeitos de ondulação do acidente da seguinte forma.

1. comece com todos os pontos em 0,0 (estado de repouso), exceto o ponto de impacto, que é definido como o valor do impacto. 2. calcule a nova posição de um ponto na superfície como a média de 1) localização atual e 2) média de vizinhos. 3. atualize todos os pontos de uma vez.

As posições mais externas do edifício são assumidas como sempre permanecendo em 0. Para simplificar, você pode assumir que seu edifício tem uma altura de 7 e uma largura de 5. Observe que é importante armazenar alterações na matriz e aplicá-las todas de uma vez no final do intervalo de tempo.

Caso contrário, os algoritmos utilizam uma mistura de dados de simulação antigos e novos, o que pode produzir resultados indesejáveis.

Entrada

A entrada consistirá de quatro números inteiros. Os dois primeiros números indicam o ponto de impacto (altura, largura). O terceiro número é a força do impacto; O número final é quantos passos de tempo da simulação devem ser exibidos.

Saída

Para cada passo de tempo da simulação, o programa deve imprimir "Time <t>"(começando no tempo 0), então o estado do edifício naquele passo de tempo. O estado do edifício pode ser impresso como uma matriz de números 7x5, com cada andar do edifício em uma linha separada. Mesmo que os valores sejam armazenados como doubles, para maior clareza o programa deve convertê-los e apenas imprimir valores inteiros.

Exemplo de Entrada 1

2 2 100 3

Exemplo de Saída 1

```
Time 0
0 0 0 0 0
0 0 0 0 0
0 0 100 0 0
0 0 0 0 0
0 0 0 0 0
0 0 0 0 0
0 0 0 0 0
Time 1
0 0 0 0 0
0 6 6 6 0
0 6 50 6 0
0 6 6 6 0
0 0 0 0 0
0 0 0 0 0
0 0 0 0 0
Time 2
0 0 0 0 0
0 7 7 7 0
0 7 28 7 0
0 7 7 7 0
0 0 1 0 0
0 0 0 0 0
0 0 0 0 0
```

Exemplo de Entrada 2

2 2 0 2

Exemplo de Saída 2

```
Time 0
0 0 0 0 0
0 0 0 0 0
0 0 0 0 0
0 0 0 0 0
0 0 0 0 0
0 0 0 0 0
0 0 0 0 0
Time 1
0 0 0 0 0
0 0 0 0 0
0 0 0 0 0
0 0 0 0 0
0 0 0 0 0
0 0 0 0 0
0 0 0 0 0
```

Problema G

Go

Arquivo fonte: go.{ c | cc | java | py }

Autor:

Recentemente, IAs que jogam Go (um jogo de tabuleiro tradicional) são bem investigadas. Seu amigo Hikaru está planejando desenvolver uma nova IA de Go incrível chamada Sai e promovê-la para a empresa F ou empresa G no futuro. Como primeiro passo, Hikaru decidiu desenvolver uma IA para 1D-Go, uma versão restrita do Go original.

Tanto no Go original quanto no 1D-Go, capturar pedras é uma estratégia importante. Hikaru pediu para você implementar uma das funções de captura.

No 1D-Go, o tabuleiro do jogo consiste em L grades dispostas em uma fileira. Um estado de 1D-go é descrito por uma string S de comprimento L . O i -ésimo caractere de S descreve a i -ésima grade como o seguinte:

- Quando o i -ésimo caractere de S é B , a i -ésima grade contém uma pedra que é colorida de preto.
- Quando o i -ésimo caractere de S é W , a i -ésima grade contém uma pedra que é colorida de branco.
- Quando o i -ésimo caractere de S é $.$, a i -ésima grade está vazia.

Máximo de pedras contínuas da mesma cor são chamadas de cadeia. Quando uma cadeia é cercada por pedras com a cor do oponente, a cadeia será capturada.

Mais precisamente, se i -ésima grade e j -ésima grades ($1 < i + 1 < jL$) contêm pedras brancas e cada grade de índice k ($i < k < j$) contém uma pedra preta, essas pedras pretas serão capturadas, e vice-versa sobre a cor.

Observe que algumas das regras do 1D-Go são bem diferentes do Go original. Algumas das intuições obtidas do Go original podem causar alguns erros.

Você recebe um estado de 1D-Go em que o próximo movimento será jogado pelo jogador com pedras brancas. O jogador pode colocar uma pedra branca em uma das células vazias. No entanto, o jogador não pode fazer uma cadeia de pedras brancas que seja cercada por pedras pretas, mesmo que simultaneamente faça algumas cadeias de pedras pretas serem cercadas. É garantido que o estado dado tenha pelo menos uma grade onde o jogador pode colocar uma pedra branca e não haja cadeias que já estejam cercadas.

Escreva um programa que calcule o número máximo de pedras pretas que podem ser capturadas pelo próximo movimento do jogador das pedras brancas.

Entrada

A entrada consiste em uma linha e tem o seguinte formato:

L S

$L(1 \leq L \leq 100)$ significa o comprimento do tabuleiro de jogo e $S(|S| = L)$ é uma string que descreve o estado de 1D-Go. O estado dado tem pelo menos uma grade onde o jogador pode colocar uma pedra branca e não há correntes que já estejam cercadas.

Saída

Produza o número máximo de pedras que podem ser capturadas no próximo movimento em uma linha.

Exemplo de Entrada 1

5 .WB..

Exemplo de Saída 1

1

Exemplo de Entrada 2

5 .WBB.

Exemplo de Saída 2

2

Exemplo de Entrada 3

6 .WB.B.

Exemplo de Saída 3

0

Problema H

Negociação Consistente

Arquivo fonte: negociacao.{ c | cc | java | py }

Autor:

Sua empresa está desenvolvendo um jogo eletrônico. Neste jogo, os jogadores podem trocar itens. Esta troca segue a regra definida pelos desenvolvedores. A regra é definida como o seguinte formato: "Os jogadores podem trocar um item A_i e x_i item B_i ". Observe que a troca pode ser feita em ambas as direções. Os itens são trocados entre os jogadores e o sistema de jogo. Portanto, os jogadores podem trocar itens qualquer número de vezes.

Às vezes, os testadores encontram bugs que uma repetição de uma sequência específica de trocas causa o incremento ilimitado de itens. Por exemplo, o seguinte conjunto de regras pode causar este bug.

1. Os jogadores podem trocar um item 1 e dois itens 2.
2. Os jogadores podem trocar um item 2 e dois itens 3.
3. Os jogadores podem trocar um item 1 e três itens 3.

Neste conjunto de regras, os jogadores podem aumentar os itens ilimitadamente. Por exemplo, os jogadores começam as negociações com um item 1. Usando as regras 1 e 2, eles podem trocá-lo por quatro itens 3. E, usando a regra 3, eles podem obter um item 1 e um item 3. Ao repetir essa sequência, a quantidade do item 3 aumenta ilimitadamente.

Esses bugs podem estragar o jogo, portanto, os desenvolvedores decidiram introduzir o sistema que impede a negociação inconsistente. Sua tarefa é escrever um programa que detecte se o conjunto de regras contém o bug ou não.

Entrada

A entrada consiste em um único caso de teste no formato abaixo.

NM

$A_1B_1x_1$

.

.

.

$A_MB_Mx_M$

A primeira linha contém dois inteiros N e M que são o número de tipos de itens e o número de regras, respectivamente ($1 \leq N \leq 100000, 1 \leq M \leq 100000$).

Cada uma das seguintes M linhas fornece a regra de negociação de que um item A_i e x_i item B_i ($1 \leq A_i, B_i \leq N, 1 \leq x_i \leq 1000000000$) pode ser trocado em ambas as direções. Não há troca entre os mesmos itens, ou seja, $A_i \neq B_i$.

Saída

Exemplo de Entrada 1

4 4
1 2 2
2 3 2
3 4 2
4 2 3

Exemplo de Saída 1

No

Exemplo de Entrada 2

4 3
1 2 7
2 3 5
4 1 2

Exemplo de Saída 2

Yes

Exemplo de Entrada 3

4 4
1 2 101
2 3 99
1 4 100
4 3 100

Exemplo de Saída 3

No